KJEI's
K J College of Engineering & Management Research, Pune
Department of Computer Engineering
Savitribai Phule Pune University

Academic Year: 2025-2026

Presentation

On

"Blockchain Based Employment Platform for Industrial and Service Workforce"

Presented By
Ms.Puram Sargam Shrikant [B400720224]
Mr.Shinde Tejas Suryakant [B400720243]
Mr.Shingde Kartik Tukaram [B400720245]
Ms.Zagade Kranti Mohan [B400720265]

Guided By
Dr. Nikita J. Kulkarni

Date:10-07-2026

Contents

Introduction

- Problem: Workers face challenges including wage theft, job fraud, platform commissions ranging from 10–20%, and delayed payments.
- Solution: A blockchain-powered platform that connects employers and workers directly. No middlemen, no hidden fees, no delays.
- Impact: Empowering workers with a secure, transparent, and fair job ecosystem where trust is built into every transaction.

Problem Statement

- In today's informal labor market, especially across industrial and service sectors, there is a serious lack of trust and transparency.
- Workers often face issues like wage theft, fake job offers, or delayed payments, simply because there’s no secure system to protect them.

Objectives

- To eliminate middlemen to reduce job placement and service costs.
- To automate and secure payments using blockchain smart contracts.
- To facilitate mutual rating to build trust between workers and employers.
- To create a verified digital identity for workers to use across multiple jobs.

Outcomes

1.Fair Job Payments

-Workers are paid on time through smart contracts with no delays or chances of wage theft.

2.Lower Fraud

-QR or OTP based check-ins along with a stake system help reduce fake job postings and false attendance.

3.Less Platform Fees

-Our platform removes middlemen and intermediaries so workers can keep their full earnings.

4.Trust Through Ratings

-Both workers and employers can rate each other which builds long term trust and credibility.

5.Verified Job Completion

-Every completed job is verified using photo proof or OTP to ensure transparency for both parties.

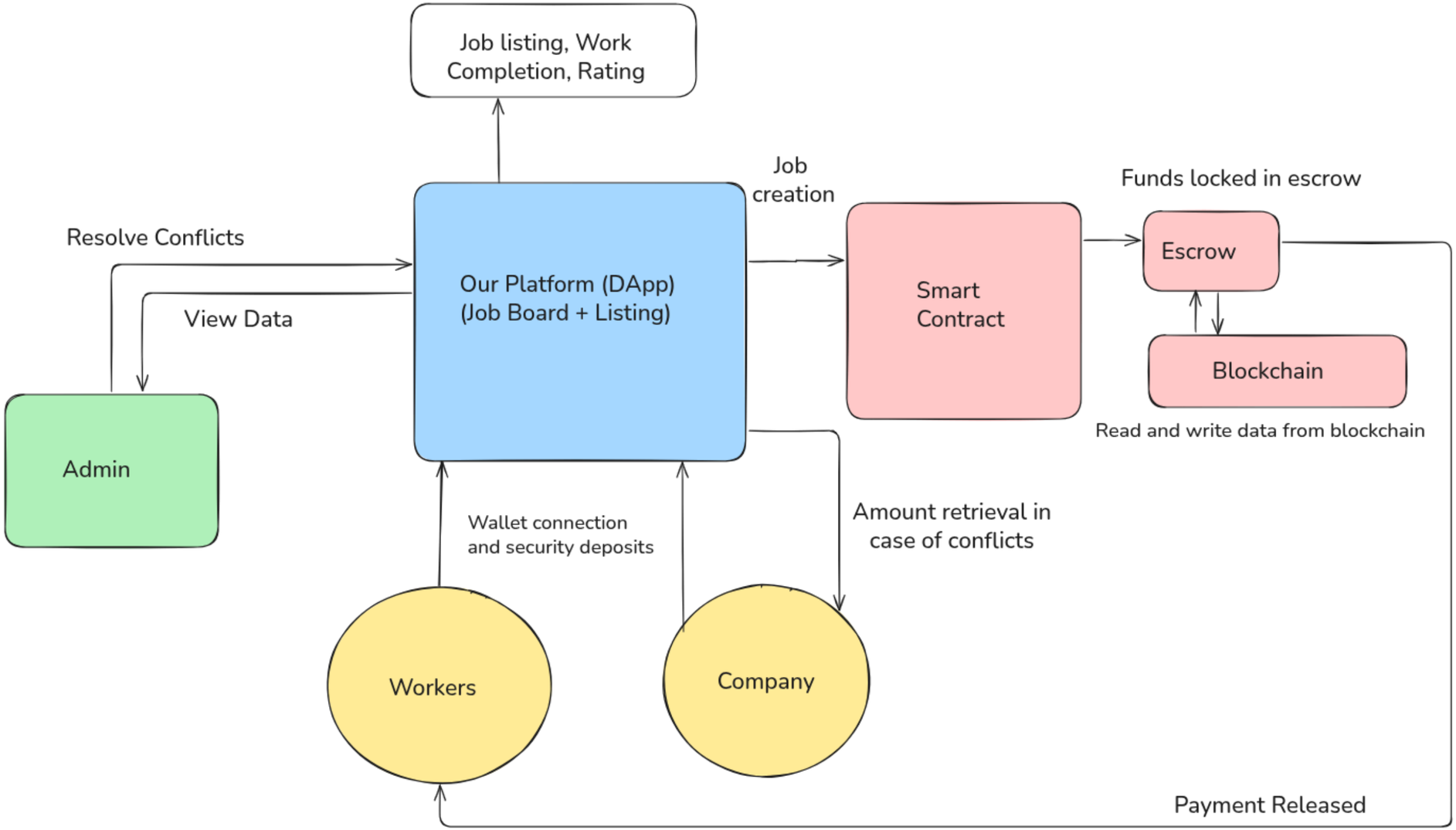
Literature Review

Paper No.	Paper Title	Name of Author/s	Methodology	Advantages	Limitations	Results
[1]	Secure Optimizations on Ethereum Bytecode Jump-Free Sequences [2025]	Elvira Albert, Samir Genaim, Daniel Kirchner, Enrique Martin-Martin	Formal verification framework using Coq proof assistant with symbolic interpreter, simplification rules, and equivalence checker for EVM bytecode optimization	Automated verification of semantic equivalence, handles memory/storage operations, prevents security vulnerabilities	Limited to jump-free code sequences, requires specific optimization patterns, computational complexity for large blocks with memory operations	Successfully verified 1,175,187 optimized blocks, found and reported bugs in existing optimizers, achieved 98.86% success rate on Solidity compiler tests
[2]	NoSneaky: A Blockchain-Based Execution Integrity Protection Scheme in Industry 4.0 [2023]	Wei-Yang Chiu, Weizhi Meng and Chunpeng Ge	Blockchain-enabled scheme ensuring execution integrity in Industry 4.0 using distributed ledgers, smart contracts, and role-based entities.	Protects against malicious behaviors and data tampering. Enhances trust and transparency in industrial workflows.	Implementation complexity in large-scale Industry 4.0 systems Performance overhead due to blockchain operations.	Validated efficiency, reliability, and practicality; showed strong protection for execution integrity in Industry 4.0 scenarios.
[3]	Leveraging Smart Contracts for Secure and Asynchronous Group Key Exchange [2023]	Victor Youdom, Yongseok Kwon , Rasheed Hussain Sunghyun Cho , and Junggab Son	Blockchain-based Group Key Smart Contract enabling decentralized and asynchronous group key exchange with processes for add/remove members, group update, and recovery.	Eliminates need for trusted third party Provides semantic security, perfect forward secrecy (PFS), and post-compromise security (PCS)	Lacks large-scale real-world validation; integration challenges with existing labor systems	Computational and storage overhead due to blockchain use Latency from sequential miner execution Scalability concerns for very large groups

Literature Review

Paper No.	Paper Title	Name of Author/s	Methodology	Advantages	Limitations	Results
[4]	Instant Function Calls Using Synchronized Cross-Blockchain Smart Contracts. [2023]	Martin Westerkamp & Axel Küpper	Proposed SmartSync framework for synchronizing smart contracts across multiple blockchains. - Uses account proof, multi-proof, and transition confirmation via Merkle root validation.	Provides instant read-only access across blockchains. - Reduces storage and execution costs by subsuming multiple transactions.	High gas costs in mainnet environments. Delay depends on storage size and mining rate. Only supports read-only synchronization.	Gas cost comparable to GPACT for single updates, but lower latency. Forking Uniswap pair contract required ~500M gas, ~442 sec.
[5]	Incentive Based Demand Response Program for Power System Flexibility Enhancement [2021]	Baraa Mohandes Nikos D. Hatziargyriou , Mohamed Shawky El Moursi	Proposed a Smart Demand Response (DR) Contract integrated with Unit Commitment (UC) problem. Incorporated Monte-Carlo simulation to handle stochastic wind-load scenarios.	Reduces RES (Renewable Energy Source) spillage significantly.- Provides flexibility in handling wind volatility.- Improves gross and net Social Welfare.	Optimization is computationally heavy (18–24 hrs for full run). Results depend strongly on wind penetration level and system setup.	Shorter commitment of thermal units and reduced ramping costs. Weekend contracts showed better SW improvement compared to weekday.
[6]	Decision Support for Blockchain Platform Selection: Three Industry Case Studies. [2020]	Siamak Farshidi , Slinger Jansen , Sergio España, Jacco Verkleij	Proposed a Decision Support System (DSS) for blockchain platform selection. Identified 71 Boolean + 4 non-Boolean features (through expert interviews, documentation.	Provides structured, knowledge-based decision-making for blockchain adoption.	Lack of standard terminology across blockchain platforms causes mapping issues	Case 1 (Finance): Hyperledger & Quorum.· Case 2 (Education): Ethereum, NEO, Hyperledger.

Proposed Methodology



Proposed Methodology

- Companies post jobs; payments are locked in escrow through smart contracts.
- Workers accept jobs, complete tasks, and get verified.
- On approval, funds are automatically released to workers.
- Conflicts are resolved by admins, with all records stored on the blockchain for trust.

Architecture & Design

The platform uses a 3-tier hybrid Web3 architecture:

- Tier 1 – Frontend (React + Vite): Wallet-connected SPA; communicates with both the backend API and Solana RPC.
- Tier 2 – Backend (Node.js / Express): Off-chain REST API; stores rich metadata in MongoDB; issues JWT tokens.
- Tier 3 – Blockchain (Solana / Anchor): On-chain escrow, job status state machine, and automated fund release.

Architecture & Design

Layer	Technology	Role
Frontend	React 18, Vite, TailwindCSS	UI for Workers, Employers & Admins
Wallet	Solana Wallet Adapter (Phantom, Solflare)	Web3 Auth & Transaction Signing
Backend API	Node.js, Express.js, JWT, Zod	Off-chain REST API, Auth, Validation
Database	MongoDB, Mongoose ODM	Off-chain profile & job metadata
Smart Contract	Rust, Anchor Framework	On-chain Escrow & Job State Machine
Blockchain	Solana (Devnet / Localnet)	Immutable transaction records

Modules / Components

#	Module	Description
1	**User Registration**	Worker & Company profile with wallet-based identity
2	**Job Posting**	Employer creates job & locks SOL into Escrow PDA
3	**Job Application**	Workers browse, filter, and apply for jobs
4	**Employment Agreement**	Smart contract governs assignment, proof & payment
5	**OTP Verification**	Start/End OTP confirms on-site attendance
6	**Dispute Resolution**	3-day window; Admin arbitration with split/favor outcomes
7	**Admin Panel**	Profile verification & dispute management dashboard
8	**Rating & Review**	Post-job ratings stored on-chain via UserRating account

Development Details

Key Modules:

A) Smart Contract (employment_contract.rs) – 6-Step Job Lifecycle:

· Step I – post_job(): Employer submits job details; SOL payment immediately transferred to Escrow PDA via CPI.

· Step 2 – assign_worker(): Employer selects a worker from applicants; Job moves to InProgress.

· Step 3 – submit_proof_of_work(): Worker submits proof (OTP/GPS/Photo); dispute_deadline set to now + 3 days.

· Step 4 – release_payment(): After dispute period expires, funds auto-released to worker wallet.

· Step 5 – create_dispute(): Employer raises dispute within 3-day window; job moves to Disputed.

Step 6 – resolve_dispute(): Admin resolves with FavorWorker / FavorEmployer / Split outcome.

Development Details

B) Backend API (Node.js / Express):

- authMiddleware: Verifies JWT Bearer tokens; populates req.user with wallet address and user type.
- adminAuthMiddleware: Validates JWT and checks wallet against hardcoded admin public key.
- jobController: Manages job creation, application approval, OTP generation/verification, proof recording, and fund transfer logging.
- adminController: Handles admin login, fetches/verifies worker and company profiles with PDA assignment.
- Validation: Zod schemas validate all API inputs; Mongoose schemas enforce DB-level constraints.

Development Details

C) Frontend (React + Vite):

- WalletContext: Global React context storing connected wallet address and verification state.
- Role-based routing: Separate dashboards for /worker/dashboard, /company/dashboard, /admin/dashboard.
- Solana Wallet Adapter: Phantom, Solflare, Torus wallets; connection to local/devnet RPC.
- useAuth hook: Handles wallet signing and backend JWT authentication flow.

Development Details

D) Database Design (MongoDB Collections)

Collection	Key Fields	Purpose
WorkerProfile	walletAddress, name, skills[], jobCategories[], rating, PDAAddress	Stores worker off-chain metadata
WorkerSkill	workerWallet, skillName, category, proficiencyLevel, experienceYears	Detailed skill records
WorkerAvailability	workerWallet, dayOfWeek, startTime, endTime	Availability schedule
CompanyProfile	walletAddress, companyName, companyType, location, rating	Employer off-chain metadata
Job	jobPDA, escrowPDA, companyWallet, title, status, applications[], proofOfWork, disputePeriod	Central job document
JobApplication	jobId, workerWallet, status, coverLetter	Worker applications

Code Snippets

A) Smart Contract – Escrow Fund Lock on Job Post (Rust):

```
// Employer posts job AND locks payment in escrow immediately
pub fn post_job(ctx: Context<PostJob>, payment_amount: u64, ...) -> Result<()> {
 let cpi_context = CpiContext::new(
 ctx.accounts.system_program.to_account_info(),
 anchor_lang::system_program::Transfer {
 from: ctx.accounts.employer.to_account_info(),
 to: escrow.to_account_info(), // PDA escrow
 },
 );
 anchor_lang::system_program::transfer(cpi_context, payment_amount)?;
 escrow.is_locked = true;
}
```

Code Snippets

B) Backend – JWT Auth Middleware (Node.js):

```
// authMiddleware.js
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = { userId: decoded.userId, walletAddress: decoded.walletAddress,
        userType: decoded.userType };
next();
```

C) Backend – Auto INR Calculation Pre-save Hook (Mongoose):

```
// jobModel.js pre-save hook
if (!this.paymentAmountINR && this.paymentAmount) {
  const SOL = this.paymentAmount / 1e9;  // lamports to SOL
  this.paymentAmountINR = Math.round(SOL * 8000); // ~8000 INR per SOL
}
```

Code Snippets

D) Frontend – Wallet Authentication Flow (React):

```
// useAuth.js
const signMessage = async () => {
  const msg = new TextEncoder().encode('Sign in to DeWages Network');
  const sig = await signMsg(msg); // Phantom wallet signs
  await axios.post('/v1/auth/signin', { walletAddress, signature });
};
```

Tools & Environment

Tool / Platform	Version	Purpose
VS Code	Latest	Primary IDE for all development
Node.js	v18+	Backend runtime environment
React + Vite	18 / 7.x	Frontend SPA framework
Rust + Anchor	0.29.0	Smart contract development
MongoDB	7.x	Off-chain database
Docker	Latest	Containerized service deployment
Solana CLI	Localnet	Blockchain test environment
Phantom Wallet	Browser Extension	Web3 wallet for testing
Postman	Latest	API testing & documentation
Jest	29.x	Automated backend testing

Validation & Testing

i. Testing Strategy

Type	Description	Files
Unit Testing	Tests individual model schemas, Zod validators, and middleware in isolation	models.test.js, schemas.test.js, middleware.test.js
Integration Testing	Tests controller functions with real in-memory MongoDB interactions	admin.test.js, job.test.js
System Testing	End-to-end workflows: full job lifecycle, dispute resolution, multi-user scenarios	workflow.test.js
User Acceptance Testing	Validates business rules: OTP flow, dispute periods, fund transfer blocking, role-based access	Covered in system & integration tests

Validation & Testing

ii. Test Cases

Test ID	Input	Expected Output	Actual Output	Status
TC-M01	Valid worker profile data	Worker created with all fields saved	Worker created, DB constraints passed	**PASS**
TC-M02	Worker data without walletAddress	Mongoose ValidationError thrown	ValidationError thrown correctly	**PASS**
TC-M28	paymentAmount = -100	ValidationError (min: 0)	ValidationError thrown	**PASS**
TC-M30	0.5 SOL, no INR provided	Auto-calc: 0.5 x 8000 = ₹4000	paymentAmountINR = 4000	**PASS**
TC-M31	3 applications (mixed status)	Virtual: total=3, pending=1, approved=1	Virtual fields computed correctly	**PASS**
TC-MW01	Valid Bearer JWT token	next() called, req.user populated	Middleware passed, user set	**PASS**
TC-MW05	Expired JWT token	401 Token expired	401 returned correctly	**PASS**
TC-MW09	Admin wallet token	next(), req.user.isAdmin = true	Admin access granted	**PASS**
TC-MW10	Non-admin wallet token	403 Admin privileges required	403 returned correctly	**PASS**

Validation & Testing

ii. Test Cases

Test ID	Input	Expected Output	Actual Output	Status
TC-J09	Duplicate worker application	Error: Worker already applied	Error thrown as expected	PASS
TC-J21	3-day dispute period set	isActive=true, 3-day diff verified	Dispute period stored correctly	PASS
TC-J22	Expired dispute period	isExpired=true, status=completed	Auto-completion logic verified	PASS
TC-A01	Valid admin credentials	200 OK + JWT token returned	Token returned	PASS
TC-A09	Verify worker (PDA + isVerified)	200, DB state updated	Worker verified in DB	PASS
TC-SYS01	Full 14-step job lifecycle	Job completed + funds transferred	All lifecycle steps passed	PASS
TC-SYS02	Employer raises dispute in window	Status = disputed, dispute stored	Dispute handled correctly	PASS
TC-SYS10	Fund transfer during active dispute	Transfer blocked	canTransferFunds = false	PASS

Validation & Testing

iii. Validation Techniques

How Correctness Was Verified:

Technique	Description
Schema Validation	Mongoose schema constraints: required fields, enum values, min/max ranges, maxlength, unique indexes.
Zod Input Validation	All API payloads validated via Zod schemas before reaching controllers. Rejects malformed inputs.
JWT Token Verification	Valid, expired, invalid, and wrong-secret tokens tested. Admin wallet checked against hardcoded pubkey.
Database State Assertions	After every operation, MongoDB queried to verify persisted state matches expected values.
Virtual Field Computation	Mongoose virtual fields (isFullyVerified, totalApplications) verified for correctness.
Pre-save Hook Verification	Auto-calculations (INR conversion, updatedAt timestamps) confirmed on every save.
Business Logic Validation	Domain rules tested: duplicate application prevention, status transitions, OTP expiry, fund lock enforcement.
Boundary Testing	Skills max=20, rating 1-5, limit max=100, page min=1, bio maxlength=500 all verified.

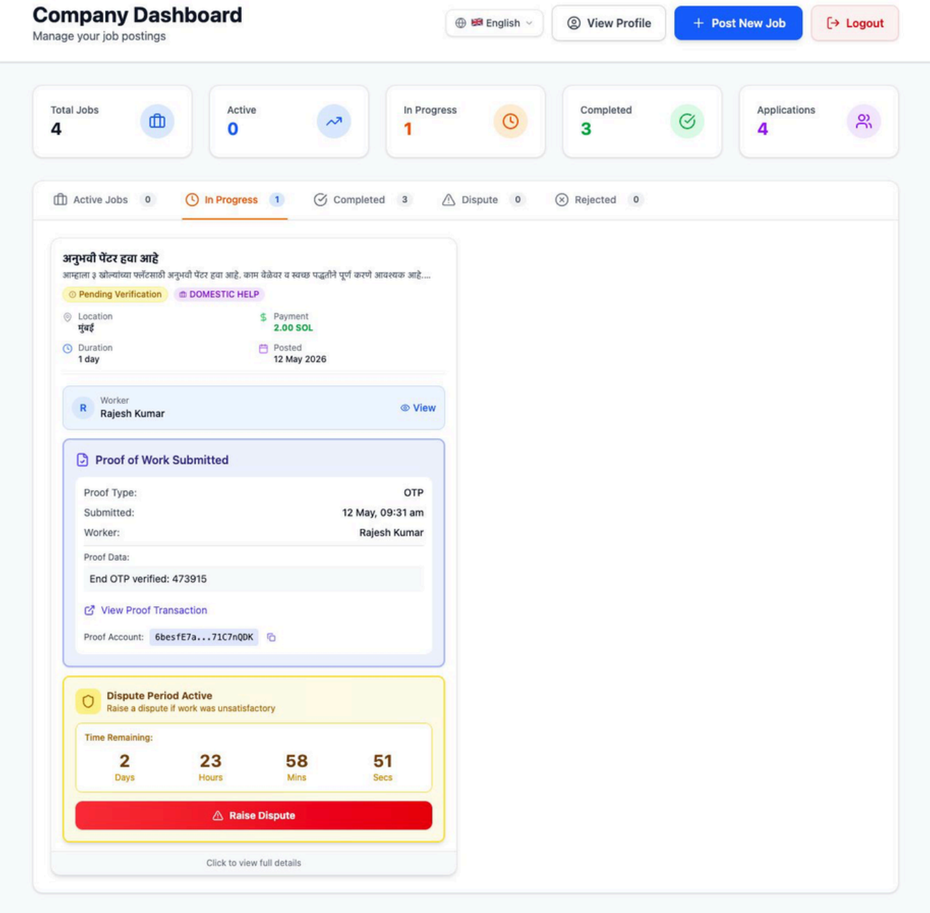
Validation & Testing

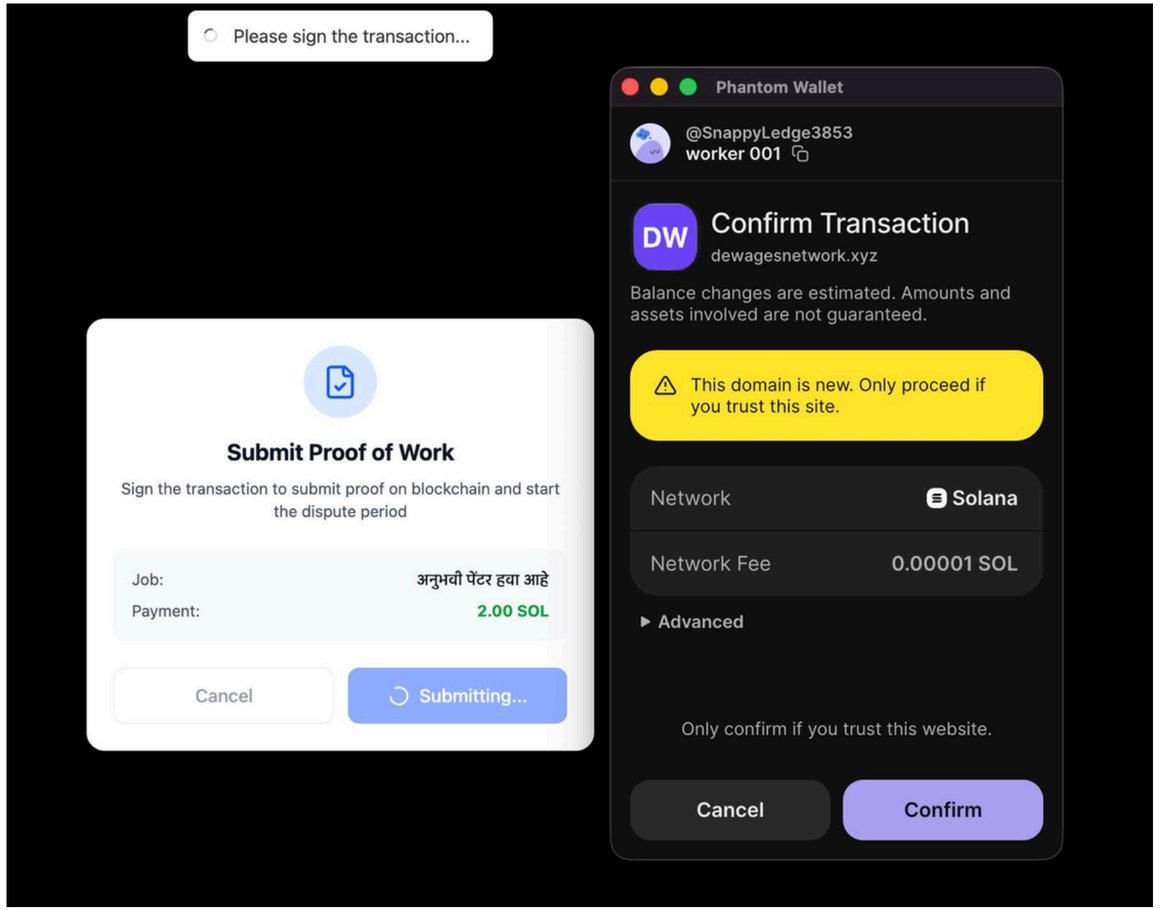
Data Validation Methods:

Method	Applied To
Required Field Validation	walletAddress (Worker/Company), jobPDA, escrowPDA, title, description, category
Enum Constraints	experienceLevel, jobCategories, companyType, status, proofType, disputeStatus
Range / Boundary Validation	rating (0-5), skills (max 20), jobCategories (max 5), pagination (page>=1, limit<=100)
Format Validation	Email (regex), phone (regex), Solana public key (Base58 format)
Uniqueness Constraints	jobPDA (unique index), JobApplication (jobId + workerWallet composite unique)
Default Value Verification	All model defaults verified: rating=0, isActive=false, status=open, fundTransfer.isTransferred=false

Results and Analysis

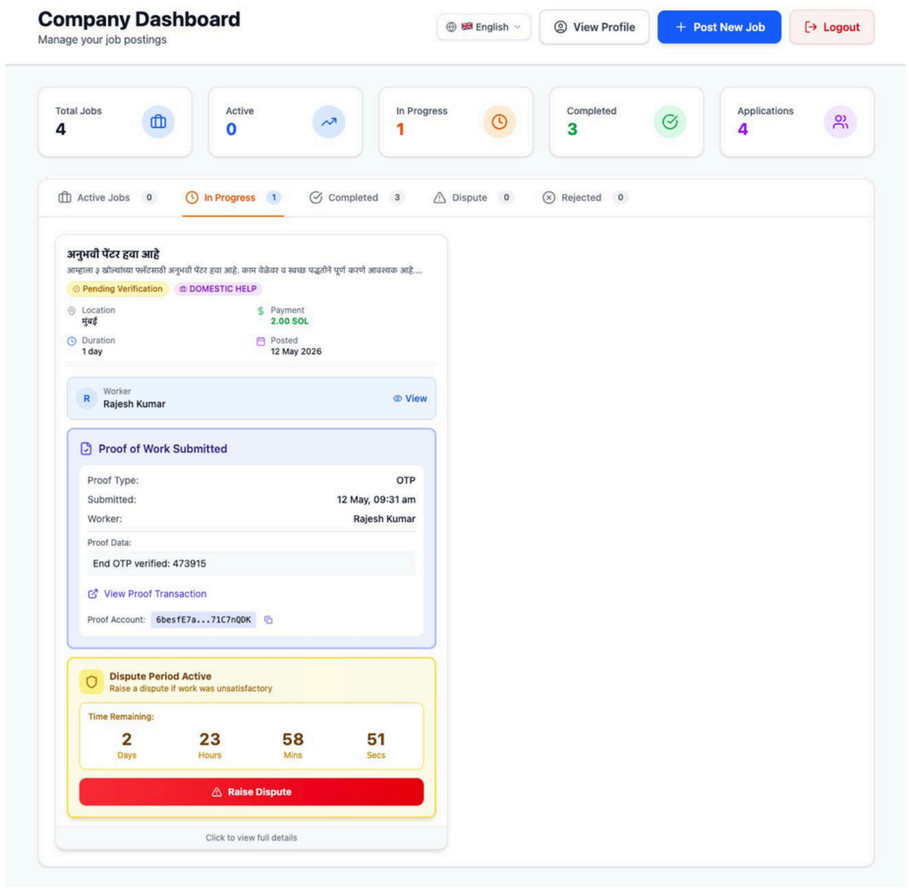
- Screen shots of output

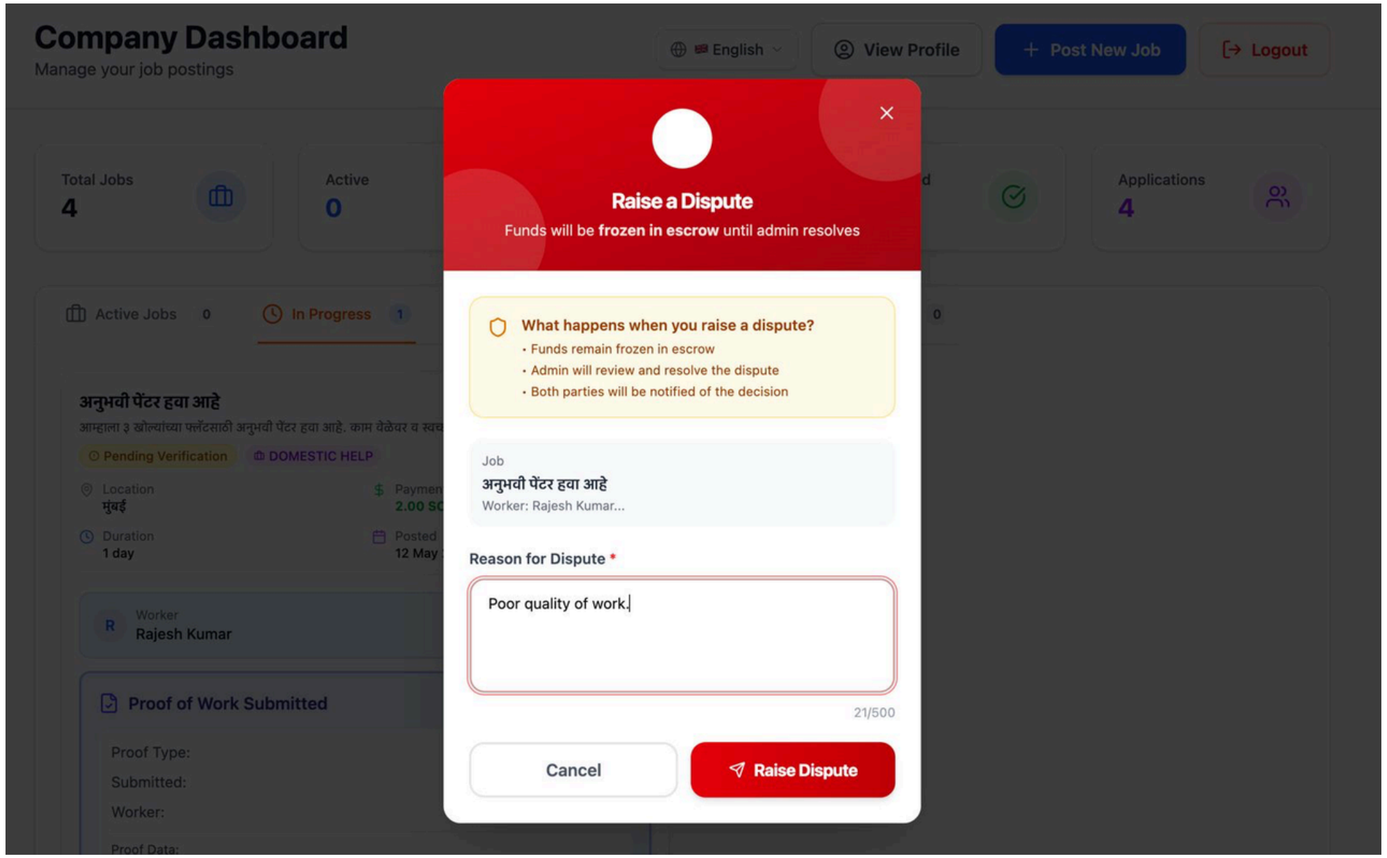




Results and Analysis

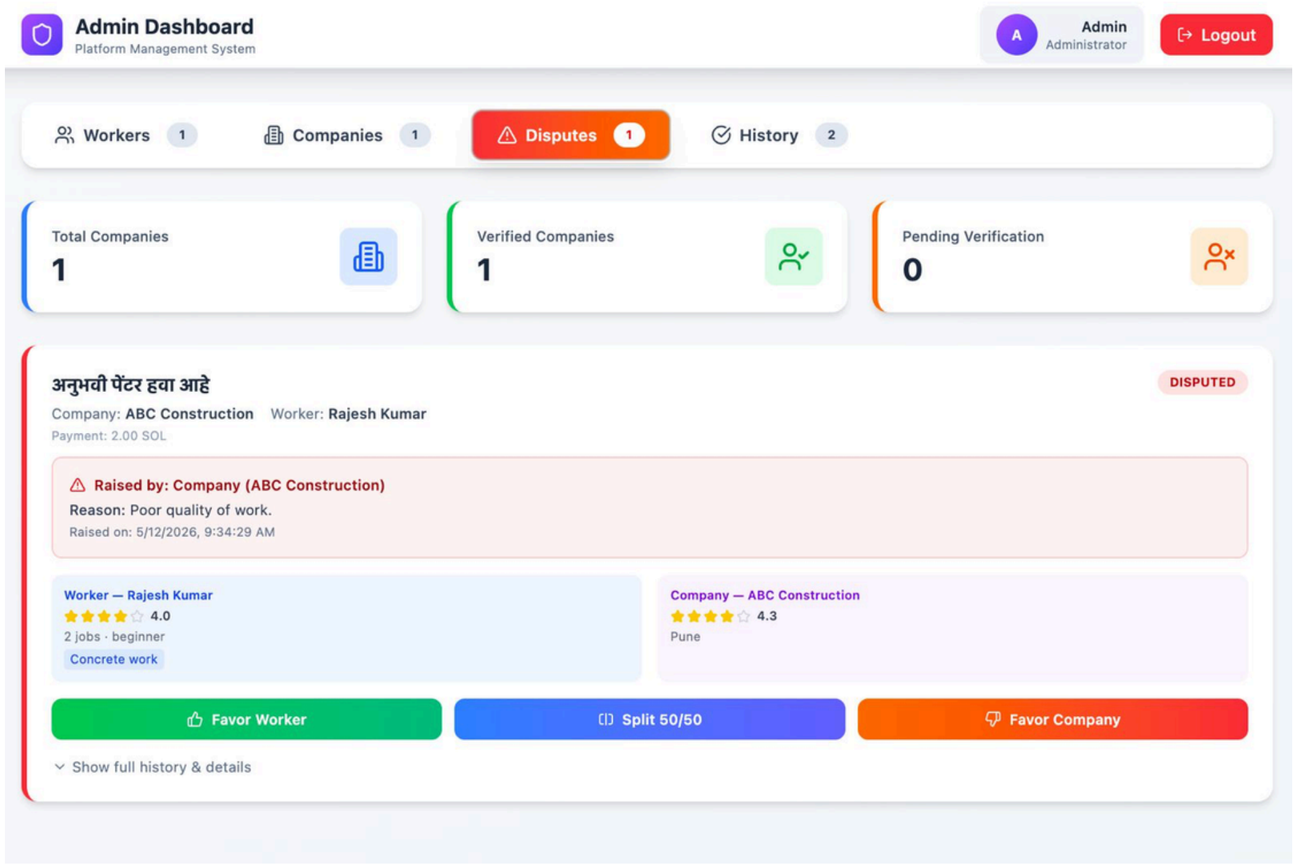
- Screen shots of output

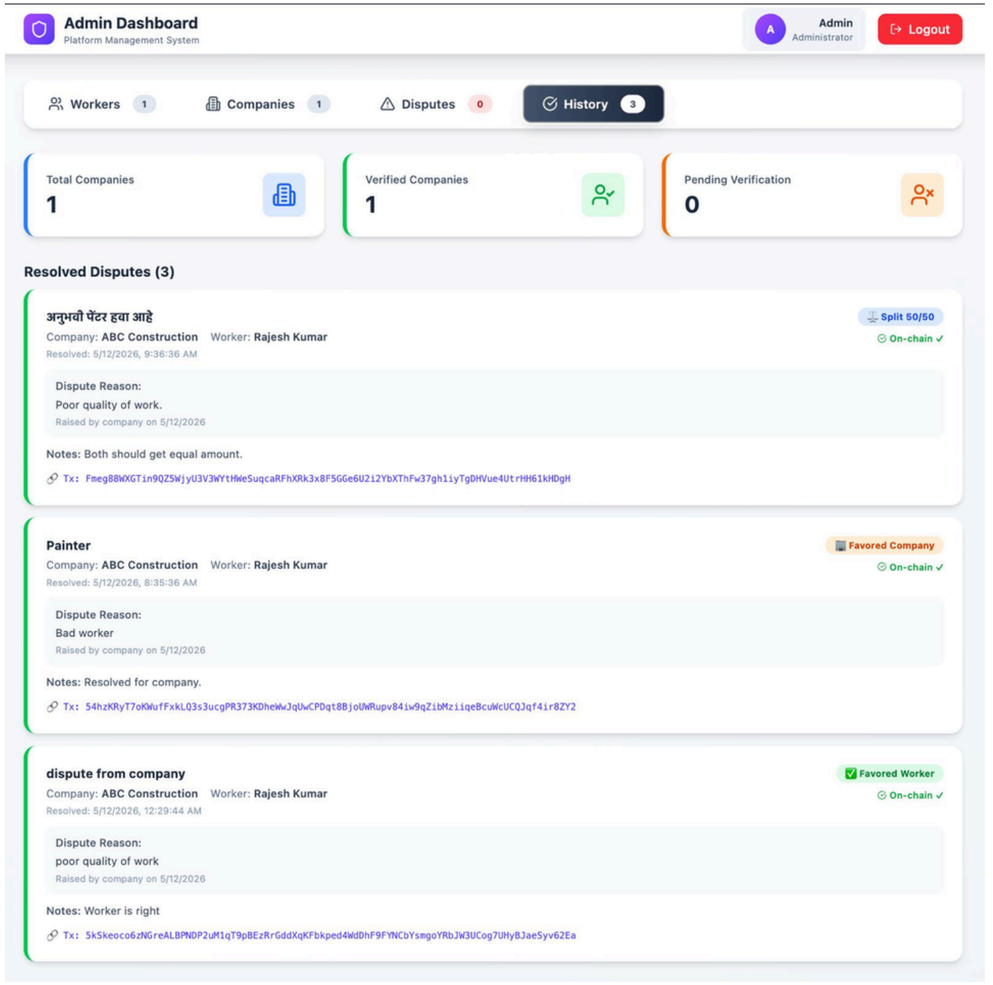




Results and Analysis

- Screen shots of output





Conclusion

- Blockchain-based platform ensures fair wages, verified work, and trust by removing middlemen and reducing exploitation.

- Leveraging smart contracts and decentralized processes, it provides transparent, secure, and timely payments while promoting accountability and reliability.

References

[1] E. Albert, S. Genaim, D. Kirchner, and E. Martin-Martin,"Secure Optimizations on Ethereum Bytecode Jump-Free Sequences,"
IEEE Transactions on Dependable and Secure Computing, vol. 22, no. 4, pp. 3676–3689, Jul./Aug. 2025.
🔗 https://ieeexplore.ieee.org/document/10870158

[2] W.-Y. Chiu, W. Meng, and C. Ge,"NoSneaky: A Blockchain-Based Execution Integrity Protection Scheme in Industry 4.0,"
IEEE Transactions on Industrial Informatics, vol. 19, no. 7, pp. 7957–7967, Jul. 2023.
🔗 https://ieeexplore.ieee.org/document/9924304

[3] V. Y. Kemmoe, Y. Kwon, R. Hussain, S. Cho, and J. Son,"Leveraging Smart Contracts for Secure and Asynchronous Group Key Exchange Without Trusted Third Party,"
IEEE Transactions on Dependable and Secure Computing, vol. 20, no. 4, pp. 3176–3192, Jul./Aug. 2023.
🔗 https://ieeexplore.ieee.org/document/9827566

[4] D. Hyang, H. Lee, H. Yu, and Y. Park,"Instant Function Calls Using Synchronized Cross-Blockchain Smart Contracts,"
IEEE Access, vol. 12, pp. 14923–14937, 2024.
🔗 https://ieeexplore.ieee.org/document/10015691

[**5]** A. Sharma, R. Kumar, B. P. Singh, and R. R. Singh,"Incentive-Based Demand Response Program for Power System Flexibility Enhancement,"
IEEE Transactions on Industry Applications, vol. 59, no. 6, pp. 6706–6714, Nov./Dec. 2023.
🔗 https://ieeexplore.ieee.org/document/9284501

[6] M. D. Zarour, M. N. Alenezi, A. Alsanad, and R. S. AlQasim,"Decision Support for Blockchain Platform Selection: Three Industry Case Studies,"
IEEE Access, vol. 9, pp. 142724–142737, 2021.
🔗 https://ieeexplore.ieee.org/document/8954801

References

[7] K. S. P. Kumar, K. S. M. Kumar, and R. Yogitha "The Evolution of Labor Market: A Blockchain Based Decentralized Hiring System for Efficient Labor Acquisition"- 2024 ICRITO Conference, Noida, India
🔗 https://ieeexplore.ieee.org/document/10522318

[8] A. Pinna & S. Ibba "A Blockchain-Based Decentralized System for Proper Handling of Temporary Employment Contracts" - Advances in Intelligent Systems and Computing (AISC), Springer, 2019
🔗 https://link.springer.com/chapter/10.1007/978-3-030-01177-2_88

[9] E. B. Sifah et al. "BEMPAS: A Decentralized Employee Performance Assessment System Based on Blockchain for Smart City Governance" - IEEE Access, Vol. 8, 2020
🔗 https://ieeexplore.ieee.org/document/9099792

[10] Ruslan Dolzhenko "Economics of Using Blockchain in the System of Labor Relations"-Advances in Intelligent Systems and Computing, Vol. 986, Springer, 2019
🔗 https://link.springer.com/chapter/10.1007/978-3-030-19813-8_28

[11] M. P. Lamela, J. Rodríguez-Molina et al. "A Blockchain-Based Decentralized Marketplace for Trustworthy Trade in Developing Countries" - IEEE Access, Vol. 10, 2022
🔗 https://ieeexplore.ieee.org/document/9843965

[**12**] Giuseppe Antonio Pierro and Roberto Tonelli, "Can Solana be the Solution to the Blockchain Scalability Problem?" – 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), Honolulu, HI, USA.
🔗 https://ieeexplore.ieee.org/document/9825807